# Isolation Forest

## 🧭 Goal of This Lesson:

You'll learn how to:

1. Prepare your data
2. Use `IsolationForest` to detect anomalies
3. Interpret the results

---

## 🛠️ Prerequisites:

You should already have:

- A clean dataframe (`df_model`)
- All features numeric (we did one-hot encoding)
- Features scaled (`X_scaled`)

We'll assume `X_scaled` is your input.

## 🔷 Step 1: Import and Initialize the Model

python Kopiuj Edytuj

```python
from sklearn.ensemble import IsolationForest

# Create the model
iso_forest = IsolationForest(
    n_estimators=100,          # number of trees
    contamination=0.02,        # roughly how many outliers we expect (2%
    random_state=42
)
```

## 🔷 Step 2: Fit the Model on the Data

python Kopiuj Edytuj

```python
# Fit the model to the scaled feature data
iso_forest.fit(X_scaled)
```

This builds an ensemble of trees, each learning to isolate unusual patterns.

## 🔷 Step 3: Predict Anomalies

python Kopiuj Edytuj

```python
# Get predictions
labels = iso_forest.predict(X_scaled)
```

This returns:

- `1` → normal
- `-1` → anomaly

To turn this into a column in your dataframe:

python Kopiuj Edytuj

```python
df_model['AnomalyLabel'] = labels
```

## 🔷 Step 4: View Anomalies

python Kopiuj Edytuj

```python
# Filter the anomalous rows
anomalies = df_model[df_model['AnomalyLabel'] == -1]

# See the top 5
anomalies.head()
```

Now you can manually inspect what makes them different:

- Are the amounts high?
- Was the time weird?
- Are login attempts suspicious?

## 🧠 How Does It Work?

- **Randomly splits features**
- Measures how *easily* each point gets isolated
- Fewer splits = more anomalous

You don't tell it what fraud looks like — it figures out what's *unusual*.

## 🔁 Try It Yourself

If you're coding along, run:

python Kopiuj Edytuj

```python
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.02, random_state=42)
iso.fit(X_scaled)
df_model['anomaly'] = iso.predict(X_scaled)
df_model[df_model['anomaly'] == -1].head()
```

## 🌲 Why “Isolation Forest”?

### It comes from a **scientific paper published in 2008:**

> **"Isolation Forest"**
> Fei Tony Liu, Kai Ming Ting, Zhi-Hua Zhou
> Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM)

The name describes **how the algorithm thinks about anomalies**.

---

## 🧠 The Core Idea:

> **Anomalies are easier to isolate.**

## 🔍 In a nutshell:

Imagine you randomly split data points (like a decision tree):

- **Normal points** are in dense clusters → it takes **many splits** to isolate them.
- **Anomalies** are off by themselves → they can be **isolated in fewer splits**.

> 💡 *The fewer splits it takes to isolate a point, the more likely it's an anomaly.*

## 🌲 Why “Forest”?

Because the algorithm builds not just one tree — but a **whole forest of random trees** (like Random Forests).
Each tree tries to randomly split the data and "trap" points.

Then it asks:

- On average, how many steps does it take to isolate this point?
- If the average is **low**, it’s **weird** (anomalous).
- If it takes many steps, it’s **normal**.

## 📊 Scientific Foundation

| Concept | Explanation |
|---|---|
| Isolation | Based on how easy it is to separate a point |
| Tree | Randomly split data recursively |
| Forest | Multiple trees give stable, averaged results |
| No density, no distance | Unlike other methods (e.g. kNN), it doesn't compute distance or density — only isolation |

It's fast, scalable, and **doesn't require assumptions about distribution** — which is why it's widely used in fraud detection, security, and health monitoring.

---

## 🧠 So in one sentence:

> The **Isolation Forest** algorithm is named for its ability to isolate anomalies quickly through random splits across a forest of trees.

# 🧠 What is K-Means?

**K-Means** is an algorithm that:

- Groups similar data points into clusters
- Finds the **“center” (mean)** of each group
- Repeats until the groups are stable

It’s called **K**-Means because:

- **K** = number of clusters you ask for
- “Means” = based on **averages** of the points in each group

## 🧭 Use Case:

> You don't know what the labels should be. You want to see **how your data naturally groups itself**.

---

# ✅ What You'll Learn

By the end of this, you'll know how to:

1. Prepare data for clustering
2. Fit `KMeans` using scikit-learn
3. Visualize the clusters (in 2D using PCA)
4. Interpret the results

# 🔧 Step-by-Step Guide

---

## 🔷 Step 1: Import the Model

python Kopiuj Edytuj

```python
from sklearn.cluster import KMeans
```

## 🔷 Step 2: Fit the Model

You choose **K** (number of clusters) up front.

python Kopiuj Edytuj

```python
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)  # X_scaled = your clean, scaled dataset
```

## 🔷 Step 3: Get Cluster Assignments

python Kopiuj Edytuj

```python
cluster_labels = kmeans.labels_  # Each point now has a cluster ID
```

Add it to your dataframe:

python Kopiuj Edytuj

```python
df_model['Cluster'] = cluster_labels
```

## 🔷 Step 4: Visualize the Clusters

Let's plot them in 2D using PCA:

python Kopiuj Edytuj

```python
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap='viridis'
plt.title('K-Means Clustering (PCA Projection)')
plt.xlabel('PC 1')
plt.ylabel('PC 2')
plt.colorbar(label='Cluster')
plt.grid(True)
plt.show()
```

## 🧠 How Does It Work?

| Step | What K-Means Does |
|---|---|
| 1 | Randomly chooses K centroids |
| 2 | Assigns each point to the closest centroid |
| 3 | Recalculates centroids based on current members |
| 4 | Repeats until clusters don’t change much |

It **doesn’t know labels** — it just finds **natural groupings**.

## ⚠️ Tips & Gotchas

| Tip | Why it matters |
|---|---|
| Scale your features | K-Means is distance-based |
| Try different K values | Use **elbow method** or **silhouette score** |
| PCA is just for plotting | K-Means runs in full feature space |

## Bonus: Want to choose the best `k`?

Try the **elbow method**:

python Kopiuj Edytuj

```python
inertias = []
for k in range(1, 10):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(range(1, 10), inertias, marker='o')
plt.title('Elbow Method For Optimal k')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.show()
```

# 🧪 Recap of What You Just Learned:

| Step | Skill |
| --- | --- |
| 🤖 KMeans | Unsupervised clustering |
| 📊 PCA + Plot | Visualization of clusters |
| 📈 Elbow method | Model selection strategy |
| 🧠 Label-free analysis | Pure data exploration |